AutoIA V1 Validation Strategy: The 381-Test Evidence Base

Version: AutoIA 1.0.1
Date: September 1, 2026

Executive summary

AutoIA V1 is a certification system, so its test suite is not intended merely to show that functions return expected-looking numbers. The suite is designed to challenge the full proof chain: exact interpretation of binary64 inputs, outward-rounded trusted primitives, dependency-bearing representations, rigorous polynomial ranging, domain and path proofs, semantic numerical operations, fail-closed behavior, architectural trust boundaries, and execution on engineering code that was not written for AutoIA.

The authoritative V1 collection contains **381 named pytest test cases in 56 test files**. During the final fresh-process 1.0.1 release run those named tests also executed **2,405 pytest subtests**, in addition to dense-grid, exact-rational, randomized, and high-precision checks performed internally by several tests. The number 381 therefore counts stable named test cases, not every individual numerical witness. The 17 tests added in 1.0.1 specifically attack the physical-input specification boundary between an intended exact-real interval and its finite binary64 proof enclosure.

All 381 named tests passed in the final grouped audit. The groups were deliberately executed in fresh Python processes because a known long-process test-harness slowdown can appear after many proof-heavy audit cases accumulate state. The same constituent tests pass promptly when isolated; the release record therefore uses bounded fresh processes and treats the monolithic-process slowdown as a documented harness/runtime limitation, not a proof failure.

Why these tests were chosen

The suite follows seven validation principles:

1. **Test the trusted base independently.** Whenever practical, proof-critical numerical kernels are compared with exact rational arithmetic, analytic extrema, high-precision calculations, or frozen independent oracles.
2. **Test containment, not visual agreement.** A rigorous interval may be conservative; the required property is that the exact value is enclosed and that domain/path claims are proved rather than guessed.
3. **Test dependency preservation.** AutoIA is valuable only if repeated uncertain quantities, polynomial dependence, and shared approximation errors remain correlated through downstream calculations.

4. **Test ordinary Python semantics.** Arrays, aliases, imported helpers, branches, SciPy calls, and cross-module organization are tested because engineering users should not have to rewrite a model into a bespoke interval language.
5. **Test failure as part of correctness.** Unsupported or unprovable operations must fail explicitly and without returning an uncertified nominal answer.
6. **Test architectural boundaries.** The soundness argument depends on keeping the trusted numerical core small and keeping controllers/application-specific theory outside it.
7. **Test adversarial engineering code.** External and independently authored examples are included specifically to expose missing language semantics and controller weaknesses that synthetic unit tests tend to miss.

Final release-run totals

Fresh-process batch	Named tests	Pytest subtests	Result
Batch 1	97	8	Passed
Batch 2	114	22	Passed
Batch 3	92	2,317	Passed
Batch 4	78	58	Passed
Total	**381**	**2,405**	**Passed**

The source suite was deliberately partitioned into fresh processes because the already documented long-process test-harness slowdown can appear after many proof-heavy audit cases accumulate state. No constituent test failed.

The suite is complemented by **27/27 source and clean-wheel numbered examples**. ExactBO/CertOpt compatibility is checked separately after the final AutoIA wheel is built because those are downstream application packages rather than part of AutoIA's 381-test proof base.

1.0.0 execution-policy regressions retained in 1.0.1

Two release-only scheduling tests raise the named-test total from the dev14 checkpoint's 362 to 364 without changing the 2,402 parameterized subtests. They verify that the high-level `workers="auto", parallel="auto"` path keeps small finite batches serial and that proposal `device="auto"` resolves conservatively to CPU. These are non-proof strategy tests: explicit parallel CPU proof and explicit MPS/CUDA proposal routes remain available, and all rigorous certificates still come from the audited CPU binary64 backend.

Real Apple hardware acceptance before the 1.0.0 freeze demonstrated 5.825x multicore proof speedup at 1,000 boxes and 11.192x at 10,000 boxes, while a light Torch proposal workload remained faster on CPU than MPS through

10,000 points. This evidence motivates automatic CPU scheduling but explicit GPU opt-in.

An external analytic-EHVI application acceptance test also exercised the public application-theorem boundary. The theorem layer reduced a coarse troublesome EHVI portfolio from `[0, 2.146489...]` to `[0.0339973, 0.0817242]`, with diagnostic dense samples `[0.0385795, 0.0719398]`. This is boundary evidence, not an AutoIA acquisition-function feature.

New 1.0.1 input-specification regressions

AutoIA 1.0.1 adds 17 named tests in `tests/test_input_specification.py` without changing ordinary model-constant semantics. These tests verify three distinct physical-input contracts: direct `Interval(lo, hi)` means exact stored binary64 endpoints; `Interval.enclosing(...)` constructs and exactly verifies a finite binary64 superset of exact decimal/rational endpoints; and `Interval.outward(...)` performs explicitly labeled one-ulp padding of stored binary64 values without claiming provenance.

The tests include positive and negative nonrepresentable decimals, exact `Fraction` and `Decimal` specifications, exactly representable endpoints, nonrepresentable point intervals, subnormal neighborhoods, scientific notation, rejection of float arguments to the exact-real constructor, nonfinite/out-of-range fail-closed cases, exact ordering checks before machine conversion, pickle/process metadata preservation, one-ulp padding at zero and finite extremes, result provenance, and prepared-cache provenance. A separate exact-rational audit, `validation/input_specification/audit_input_specification.py`, checks 10,081 endpoint containments, including 10,000 seeded randomized rational intervals.

The implementation deliberately treats binary64 conversion as a proposal and verifies the final containment against exact rational arithmetic. The trusted binary64 environment and adjacent-value stepping are audited; inability to prove a finite enclosing endpoint fails closed.

Test-family map

Family	Named tests	Purpose
1. Trusted arithmetic, reductions, and proof-kernel exactness	**45**	Directly challenges the small numerical layer on which machine-rigorous containment rests.

Family	Named tests	Purpose
2. Core enclosures, polynomial algebra, and shared-dependence representations	**33**	Checks the central dependency-bearing mathematical representations and their architectural isolation.
3. Polynomial ranging, monotonicity, Bernstein conversion, and performance-oriented ranging	**35**	Checks how retained polynomial dependence is converted into rigorous scalar ranges without allowing faster kernels to change the proof.
4. Elementary, power, special, and normal-distribution mathematics	**45**	Covers nonlinear functions and their mathematical domains across ordinary NumPy/math/SciPy syntax.
5. Arrays, matrix operations, SPD matrix families, and validated linear-system semantics	**41**	Covers the array-centric and linear-algebra patterns typical of engineering codes while retaining uncertainty dependence.
6. Paths, closed-boundary comparisons, domain proofs, subdivision, and automatic control	**63**	Challenges uncertain control flow, exact boundary semantics, proof-driven subdivision, and zero-knob policy behavior.
7. Local semantic dispatch and supported/recognized SciPy semantics	**43**	Tests ordinary cross-module scientific Python calls, implicit solves, interpolation, and explicit recognized boundaries.

Family	Named tests	Purpose
8. Public API, input specification, many-box/accelerator execution, application theorems, package boundaries, and typed failures	**66**	Validates the physical-input theorem boundary, user-facing contracts, repeated certification workflows, application extension mechanisms, separation, and fail-closed semantics.
9. External and independently authored engineering validation	**10**	Challenges the system with code/data patterns not designed to flatter or simplify AutoIA.

The family totals sum exactly to **381**.

1. Trusted arithmetic, reductions, and proof-kernel exactness

Directly challenges the small numerical layer on which machine-rigorous containment rests.

`test_scalar_binary64_exact.py` - 8 tests

Checks exact binary64 endpoint interpretation and scalar trusted operations against exact rational meaning. Chosen because an enclosure system cannot be machine-rigorous if the initial floating-point inputs or scalar rounding primitives are misunderstood.

Collected tests:

- `test_scalar_binary64_exact.py::TestBinary64Environment::test_environment_audit_passes_c`
- `test_scalar_binary64_exact.py::TestBinary64Environment::test_integer_interval_contains_`
- `test_scalar_binary64_exact.py::TestBinary64Environment::test_trusted_interval_canonical`
- `test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_division_by_interval_`
- `test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_edge_case_point_opera`
- `test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_random_interval_opera`
- `test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_random_raw_binary64_p`
- `test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_signed_zero_and_under`

`test_trusted_boundary.py` - 2 tests

Enforces the architectural boundary around the trusted kernel. Chosen to keep the trusted computing base small and prevent higher-level heuristic/controller code from becoming part of the proof argument.

Collected tests:

- `test_trusted_boundary.py::TestTrustedBoundary::test_proof_critical_nextafter_is_confine`
- `test_trusted_boundary.py::TestTrustedBoundary::test_trusted_modules_do_not_import_highe`

`test_trusted_kernel_regression.py` - 6 tests

Regression-checks proof-critical trusted arithmetic against frozen reference behavior. Chosen because subtle rounding changes can silently invalidate every higher layer even when numerical answers look plausible.

Collected tests:

- `test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_certificate_matrix`
- `test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_integer_interval_i`
- `test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_interval_linear_al`
- `test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_nonnegative_reduct`
- `test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_point_reductions_a`
- `test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_scalar_binary64_ba`

`test_trusted_elementary.py` - 4 tests

Audits directed-rounding elementary-function kernels. Chosen because transcendental functions are common and their endpoint rounding is part of the formal containment guarantee.

Collected tests:

- `test_trusted_elementary.py::TestTrustedElementary::test_edge_point_values`
- `test_trusted_elementary.py::TestTrustedElementary::test_interval_extensions_contain_sam`
- `test_trusted_elementary.py::TestTrustedElementary::test_pi_and_ln2_constants_contain_hi`
- `test_trusted_elementary.py::TestTrustedElementary::test_random_points_against_high_prec`

`test_trusted_array_elementary.py` - 5 tests

Extends trusted elementary checks to array-valued operations and vectorized paths. Chosen because vectorization must not change the proof semantics relative to scalar execution.

Collected tests:

- `test_trusted_array_elementary.py::TestTrustedArrayElementary::test_elementwise_arithmet`
- `test_trusted_array_elementary.py::TestTrustedArrayElementary::test_exp_interval_arrays_`
- `test_trusted_array_elementary.py::TestTrustedArrayElementary::test_exp_point_arrays_hig`
- `test_trusted_array_elementary.py::TestTrustedArrayElementary::test_integer_cache_does_n`

- test_trusted_array_elementary.py::TestTrustedArrayElementary::test_sqrt_point_arrays_hi

test_trusted_power.py - 4 tests

Checks trusted constant-power primitives and domain handling. Chosen because positive, negative, integer, and fractional powers have different mathematical domains and rounding risks.

Collected tests:

- test_trusted_power.py::TestTrustedPower::test_integer_negative_base_and_domain_failures
- test_trusted_power.py::TestTrustedPower::test_interval_powers_contain_samples
- test_trusted_power.py::TestTrustedPower::test_point_powers_against_high_precision
- test_trusted_power.py::TestTrustedPower::test_random_positive_point_powers

test_reductions_exact.py - 7 tests

Uses independent exact-rational calculations for dot products, matrix products, and interval extrema. Chosen to challenge cancellation and long reductions where ordinary floating-point summation is most dangerous.

Collected tests:

- test_reductions_exact.py::TestExactReductions::test_adversarial_cancellation_dot_produc
- test_reductions_exact.py::TestExactReductions::test_certificate_matrix_helpers_against_
- test_reductions_exact.py::TestExactReductions::test_interval_matvec_against_exact_extre
- test_reductions_exact.py::TestExactReductions::test_nonnegative_upper_reductions_agains
- test_reductions_exact.py::TestExactReductions::test_point_matrix_products_against_exact
- test_reductions_exact.py::TestExactReductions::test_point_times_interval_reductions_aga
- test_reductions_exact.py::TestExactReductions::test_random_dot_products_against_exact_r

test_batched_triangular_interval.py - 4 tests

Checks interval triangular solves over batches and exact-rational references. Chosen because triangular solves are proof-critical building blocks for parameter-dependent linear and nonlinear certification.

Collected tests:

- test_batched_triangular_interval.py::TestBatchedTriangularIntervalSolve::test_arbitrary
- test_batched_triangular_interval.py::TestBatchedTriangularIntervalSolve::test_point_rhs
- test_batched_triangular_interval.py::TestBatchedTriangularIntervalSolve::test_random_ba
- test_batched_triangular_interval.py::TestBatchedTriangularIntervalSolve::test_rejects_i

test_performance_kernels.py - 5 tests

Checks faster vectorized reductions and batched proposal factorizations against rigorous references. Chosen to ensure performance engineering cannot trade away containment.

Collected tests:

- test_performance_kernels.py::TestVectorizedTrustedReductions::test_auto_mode_keeps_smal
- test_performance_kernels.py::TestVectorizedTrustedReductions::test_public_backend_reduc
- test_performance_kernels.py::TestVectorizedTrustedReductions::test_vectorized_dot_conta
- test_performance_kernels.py::TestVectorizedTrustedReductions::test_vectorized_nonnegati
- test_performance_kernels.py::TestBatchedProposalFactorization::test_polynomial_predicto

2. Core enclosures, polynomial algebra, and shared-dependence representations

Checks the central dependency-bearing mathematical representations and their architectural isolation.

test_enclosure_boundary.py - 3 tests

Checks that the enclosure layer is representation-abstract and does not depend upward on API/controller machinery. Chosen to preserve a clean proof architecture and permit additional representations safely.

Collected tests:

- test_enclosure_boundary.py::TestEnclosureBoundary::test_enclosure_modules_do_not_import
- test_enclosure_boundary.py::TestEnclosureBoundary::test_public_representations_facade_u
- test_enclosure_boundary.py::TestEnclosureBoundary::test_validated_value_has_no_concrete

test_enclosure_regression.py - 10 tests

Bitwise/reference regression tests for arithmetic, reciprocal, division, promotion, and piecewise hulls across representations. Chosen to detect semantic drift in the central dependency-bearing value object.

Collected tests:

- test_enclosure_regression.py::TestEnclosureRegression::test_interval_enclosure_matches_
- test_enclosure_regression.py::TestEnclosureRegression::test_mixed_interval_polynomial_p
- test_enclosure_regression.py::TestEnclosureRegression::test_parameter_and_generic_arith
- test_enclosure_regression.py::TestEnclosureRegression::test_parameterized_alias_is_exac
- test_enclosure_regression.py::TestEnclosureRegression::test_piecewise_hull_matches_orac
- test_enclosure_regression.py::TestEnclosureRegression::test_reciprocal_and_division_mat
- test_enclosure_regression.py::TestEnclosureRegression::test_validated_value_interval_pa
- test_enclosure_regression.py::TestEnclosureRegression::test_validated_value_polynomial_
- test_enclosure_regression.py::TestRepresentationContract::test_mixed_representation_use
- test_enclosure_regression.py::TestRepresentationContract::test_validated_value_accepts_

test_polynomial_boundary.py - 2 tests

Enforces the dependency direction of the polynomial layer. Chosen because polynomial algebra and ranging are proof machinery and must remain indepen-

dent of higher-level orchestration.

Collected tests:

- `test_polynomial_boundary.py::TestPolynomialBoundary::test_polynomial_modules_do_not_imp`
- `test_polynomial_boundary.py::TestPolynomialBoundary::test_ranging_depends_on_algebra_an`

`test_polynomial_regression.py` - 8 tests

Checks multi-index algebra, monomial bounds, multiplication, reciprocal predictors, derivatives, monotonicity, Bernstein reduction, and cost estimators against historical oracles. Chosen because these operations determine both dependency retention and polynomial-range soundness.

Collected tests:

- `test_polynomial_regression.py::TestPolynomialRegression::test_degree_and_reduced_cost_e`
- `test_polynomial_regression.py::TestPolynomialRegression::test_derivative_and_monotonici`
- `test_polynomial_regression.py::TestPolynomialRegression::test_monomial_bounds_are_bit_i`
- `test_polynomial_regression.py::TestPolynomialRegression::test_multiindex_and_cleaning_m`
- `test_polynomial_regression.py::TestPolynomialRegression::test_polynomial_addition_and_m`
- `test_polynomial_regression.py::TestPolynomialRegression::test_primary_range_methods_are`
- `test_polynomial_regression.py::TestPolynomialRegression::test_reciprocal_predictor_is_b`
- `test_polynomial_regression.py::TestPolynomialRegression::test_reduced_bernstein_is_bit_`

`test_generator_boundary.py` - 2 tests

Checks that the shared-generator representation is a proper public representation with no upward architectural dependencies. Chosen to keep correlated remainder semantics general rather than application-specific.

Collected tests:

- `test_generator_boundary.py::TestGeneratorBoundary::test_generator_layer_does_not_import`
- `test_generator_boundary.py::TestGeneratorBoundary::test_generator_representation_is_pub`

`test_shared_generators.py` - 8 tests

Checks generator identity, reuse, cancellation, arithmetic, and enclosure behavior. Chosen because shared approximation-error correlation is valuable only if provenance is preserved exactly through downstream expressions.

Collected tests:

- `test_shared_generators.py::TestSharedGeneratorRepresentation::test_elementary_function_`
- `test_shared_generators.py::TestSharedGeneratorRepresentation::test_generator_matrix_exp`
- `test_shared_generators.py::TestSharedGeneratorRepresentation::test_path_aware_execution`
- `test_shared_generators.py::TestSharedGeneratorRepresentation::test_public_representatio`
- `test_shared_generators.py::TestSharedGeneratorRepresentation::test_recomputed_expressio`
- `test_shared_generators.py::TestSharedGeneratorRepresentation::test_sampled_nonlinear_co`

- test_shared_generators.py::TestSharedGeneratorRepresentation::test_semantic_solve_retur
- test_shared_generators.py::TestGeneratorAlgebraDirect::test_manual_shared_generator_pro

3. Polynomial ranging, monotonicity, Bernstein conversion, and performance-oriented ranging

Checks how retained polynomial dependence is converted into rigorous scalar ranges without allowing faster kernels to change the proof.

test_packed_polynomial_ranging.py - 6 tests

Checks packed/batched termwise polynomial ranging and many-box axes. Chosen because performance-oriented packed representations must preserve the scalar rigorous contract.

Collected tests:

- test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_family_axis_can_in
- test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_interval_family_ma
- test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_interval_family_te
- test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_interval_minus_rep
- test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_plan_is_canonical_
- test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_point_family_termw

test_packed_polynomial_monotonicity.py - 7 tests

Checks derivative plans, monotonicity signs, and fixed-coordinate ranges against exact/scalar references. Chosen because monotonicity can sharply tighten bounds but an incorrect sign proof would be unsound.

Collected tests:

- test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_batched_
- test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_derivati
- test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_hand_con
- test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_monotoni
- test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_packed_f
- test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_portfoli
- test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_semantic

test_packed_polynomial_bernstein.py - 8 tests

Checks exact-to-enclosed Bernstein basis conversion, full and reduced coefficient families, and batched behavior. Chosen because Bernstein ranging is one of the strongest polynomial range methods and must be basis-conversion proof, not numerical fitting.

Collected tests:

- test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_dense_structur
- test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_family_bernste
- test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_fixed_dimensio
- test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_full_coefficie
- test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_full_family_ra
- test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_one_dimensiona
- test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_portfolio_pres
- test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_reduced_family

test_interval_reduction_performance.py - 7 tests

Checks vectorized point-interval and interval-interval reductions, including long products and batch dimensions. Chosen to validate the optimized numerical paths used heavily by realistic engineering arrays.

Collected tests:

- test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_auto_mod
- test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_batched_
- test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_batched_
- test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_interval
- test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_long_int
- test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_long_poi
- test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_matrix_e

test_split_scoring.py - 7 tests

Checks subdivision scoring/selection logic. Chosen because subdivision decisions affect efficiency and sharpness; tests ensure the heuristic is deterministic/consistent without becoming part of soundness.

Collected tests:

- test_split_scoring.py::TestCheapSplitScore::test_cheap_controller_certifies_only_select
- test_split_scoring.py::TestCheapSplitScore::test_cheap_controller_matches_reference_on_
- test_split_scoring.py::TestCheapSplitScore::test_cheap_controller_supports_generator_re
- test_split_scoring.py::TestCheapSplitScore::test_default_subdivision_policy_uses_cheap_
- test_split_scoring.py::TestCheapSplitScore::test_hidden_monotone_score_predicts_resolut
- test_split_scoring.py::TestCheapSplitScore::test_invalid_split_method_is_typed_public_f
- test_split_scoring.py::TestCheapSplitScore::test_stationary_score_predicts_local_progre

4. Elementary, power, special, and normal-distribution mathematics

Covers nonlinear functions and their mathematical domains across ordinary NumPy/math/SciPy syntax.

`test_elementary_boundary.py` - 2 tests

Enforces architecture boundaries for the elementary-function layer. Chosen so elementary proofs depend only on trusted machinery rather than semantic dispatch or controller logic.

Collected tests:

- `test_elementary_boundary.py::TestElementaryBoundary::test_elementary_core_depends_only_`
- `test_elementary_boundary.py::TestElementaryBoundary::test_enclosure_elementary_does_not`

`test_elementary_execution.py` - 14 tests

End-to-end tests for exp, log, log10, sqrt, trigonometric-like supported paths, composition, aliases, domains, and multivariate inputs. Chosen to exercise ordinary user syntax and real domain failures through the complete execution stack.

Collected tests:

- `test_elementary_execution.py::TestElementaryExecution::test_all_five_polynomial_functio`
- `test_elementary_execution.py::TestElementaryExecution::test_composed_exp_log_contains_i`
- `test_elementary_execution.py::TestElementaryExecution::test_direct_numpy_alias_is_dispa`
- `test_elementary_execution.py::TestElementaryExecution::test_elementary_dispatch_does_no`
- `test_elementary_execution.py::TestElementaryExecution::test_elementary_postprocessing_a`
- `test_elementary_execution.py::TestElementaryExecution::test_interval_representation_sup`
- `test_elementary_execution.py::TestElementaryExecution::test_log10_domain_failure_is_fai`
- `test_elementary_execution.py::TestElementaryExecution::test_log10_module_numpy_and_dire`
- `test_elementary_execution.py::TestElementaryExecution::test_math_module_and_direct_alia`
- `test_elementary_execution.py::TestElementaryExecution::test_multivariate_nonlinear_argu`
- `test_elementary_execution.py::TestElementaryExecution::test_object_array_ufunc_dispatch`
- `test_elementary_execution.py::TestElementaryExecution::test_polynomial_boundary_domain_`
- `test_elementary_execution.py::TestElementaryExecution::test_real_domain_failures_are_ce`
- `test_elementary_execution.py::TestElementaryExecution::test_sqrt_touching_zero_is_sound`

`test_special_functions.py` - 9 tests

Checks supported special-function semantics such as erf and standard-normal building blocks. Chosen because these functions appear in engineering statistics and application-theorem calculations and require dedicated validated approximations.

Collected tests:

- `test_special_functions.py::TestTrustedSpecialFunctions::test_erf_interval_is_monotone_e`
- `test_special_functions.py::TestTrustedSpecialFunctions::test_erf_points_contain_referen`
- `test_special_functions.py::TestTrustedSpecialFunctions::test_normal_point_functions_con`
- `test_special_functions.py::TestSpecialExecution::test_erf_polynomial_dense_containment`
- `test_special_functions.py::TestSpecialExecution::test_interval_representation`
- `test_special_functions.py::TestSpecialExecution::test_math_erf_dispatch`

- test_special_functions.py::TestSpecialExecution::test_parameterized_normal_helpers
- test_special_functions.py::TestSpecialExecution::test_standard_normal_cdf_polynomial_de
- test_special_functions.py::TestSpecialExecution::test_standard_normal_pdf_polynomial_de

test_scipy_normal_compatibility.py - 11 tests

Checks SciPy normal PDF/CDF compatibility across module, frozen distribution, and newer API forms. Chosen to demonstrate semantic compatibility with common scientific Python idioms without globally patching SciPy.

Collected tests:

- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_direct_import_al
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_frozen_norm_crea
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_frozen_norm_obje
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_imported_stats_a
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_new_normal_distr
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_preexisting_new_
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_real_scipy_objec
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_scipy_entry_poin
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_top_level_scipy_
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_uncertain_distri
- test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_vectorized_scipy

test_power_execution.py - 9 tests

End-to-end power tests across integer/fractional/negative exponents, NumPy/math aliases, zero-boundary domains, and unsupported uncertain exponents. Chosen because power syntax is ubiquitous and domain-sensitive.

Collected tests:

- test_power_execution.py::TestPowerExecution::test_domain_failures
- test_power_execution.py::TestPowerExecution::test_fractional_positive_and_negative_powe
- test_power_execution.py::TestPowerExecution::test_half_power_reuses_sqrt_quality
- test_power_execution.py::TestPowerExecution::test_interval_even_power_preserves_same_ba
- test_power_execution.py::TestPowerExecution::test_math_pow_and_alias
- test_power_execution.py::TestPowerExecution::test_negative_integer_powers
- test_power_execution.py::TestPowerExecution::test_numpy_power_and_alias
- test_power_execution.py::TestPowerExecution::test_power_dispatch_does_not_mutate_numpy_
- test_power_execution.py::TestPowerExecution::test_uncertain_exponent_unsupported

5. Arrays, matrix operations, SPD matrix families, and validated linear-system semantics

Covers the array-centric and structured linear-algebra patterns typical of engineering codes while retaining common uncertain-parameter dependence. The

SPD-family tests were added after GP MLE experiments exposed a general failure mode of entrywise intervalization: independent matrix-entry intervals admit fictitious matrices that cannot arise from one common physical input.

`test_array_semantics.py` - 16 tests

Covers array construction, mutation, broadcasting, reshape/stack/transpose, reductions, dot/matmul/inner/outer/trace, norms, and output shape. Chosen because ordinary engineering Python is array-centric and object-dtype leakage is a common failure mode.

Collected tests:

- `test_array_semantics.py::TestArrayConstructionAndMutation::test_array_constructor_and_s`
- `test_array_semantics.py::TestArrayConstructionAndMutation::test_direct_zeros_alias_is_p`
- `test_array_semantics.py::TestArrayConstructionAndMutation::test_zero_initialized_linear`
- `test_array_semantics.py::TestArrayConstructionAndMutation::test_zeros_matrix_assembly_w`
- `test_array_semantics.py::TestBroadcastAndStructuralArraySemantics::test_preexisting_num`
- `test_array_semantics.py::TestBroadcastAndStructuralArraySemantics::test_stack_concatena`
- `test_array_semantics.py::TestBroadcastAndStructuralArraySemantics::test_two_dimensional`
- `test_array_semantics.py::TestReductionsAndLinearAlgebra::test_direct_dot_alias`
- `test_array_semantics.py::TestReductionsAndLinearAlgebra::test_direct_sum_alias`
- `test_array_semantics.py::TestReductionsAndLinearAlgebra::test_dot_matmul_inner_outer_an`
- `test_array_semantics.py::TestReductionsAndLinearAlgebra::test_sum_mean_prod_axes_and_ar`
- `test_array_semantics.py::TestNormsAndResultShape::test_matrix_frobenius_norm`
- `test_array_semantics.py::TestNormsAndResultShape::test_matrix_spectral_norm_is_explicit`
- `test_array_semantics.py::TestNormsAndResultShape::test_scalar_output_shape_metadata`
- `test_array_semantics.py::TestNormsAndResultShape::test_vector_norm_and_alias`
- `test_array_semantics.py::TestNormsAndResultShape::test_vector_norm_crossing_zero_uses_s`

`test_semantic_solve_regression.py` - 5 tests

Regression-checks semantic linear solve behavior. Chosen to ensure the high-level meaning of solve(A,b) remains rigorous as lower-level representations and solver optimizations evolve.

Collected tests:

- `test_semantic_solve_regression.py::TestSemanticSolveRegression::test_componentwise_corr`
- `test_semantic_solve_regression.py::TestSemanticSolveRegression::test_interval_semantic_`
- `test_semantic_solve_regression.py::TestSemanticSolveRegression::test_polynomial_semanti`
- `test_semantic_solve_regression.py::TestSemanticSolveRegression::test_predictor_coeffici`
- `test_semantic_solve_regression.py::TestSemanticSolveRegression::test_residual_interval_`

`test_step5_general_solve.py` - 5 tests

Checks the general parameter-dependent linear solve extension beyond simple triangular/special cases. Chosen because matrix solves are a foundational engineering semantic.

Collected tests:

- test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_elementary_generated
- test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_general_recurrence_c
- test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_midpoint_preconditio
- test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_multiple_right_hand_
- test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_quadratic_2x2_matrix

test_generator_aware_solve.py - 6 tests

Checks that shared generators in matrices/right-hand sides survive solves and retain cancellation/provenance. Chosen to prevent the linear solver from becoming an irreversible dependency-destroying boundary.

Collected tests:

- test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_direct_auxiliary_g
- test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_generator_in_matri
- test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_multiple_rhs_prese
- test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_remaining_box_corr
- test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_rhs_generator_surv
- test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_shared_rhs_generat

test_spd_matrix_semantics.py - 6 tests

Validates the general parameter-dependent symmetric positive-definite matrix-family semantics: structured SPD proof, `slogdet`, determinant, and inverse quadratic forms; exact structural symmetry requirements; application-theorem access; an RBF-like MLE family; and the crucial fallback showing that the structured SPD inverse-series proof remains valid even when the older generic linear-solve contraction theorem is deliberately forced to fail. Chosen because the MLE stress test exposed a general dependency-preservation requirement that also occurs in stiffness matrices, covariance models, information matrices, Kalman filtering, and other engineering calculations.

Collected tests:

- test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_application_context_spd_logd
- test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_mle_like_family_certifies_on
- test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_non_symmetric_family_fails_c
- test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_quad_solve_spd_does_not_requ
- test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_separately_constructed_symme
- test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_slogdet_quad_solve_and_det_c

test_spd_matrix_high_precision.py - 3 tests

Audits the SPD-family theorems against independent 80-digit `mpmath` calculations for both a small analytic SPD family and an RBF-like covariance family, and verifies shrinking-box convergence. Chosen to test the new preconditioned

eigenvalue, matrix-log-series, inverse-series, spectral fallback, and tail bounds independently of ordinary binary64 sampling. The standalone audit associated with this file performs 972 high-precision scalar containment checks.

Collected tests:

- `test_spd_matrix_high_precision.py::TestSPDMatrixHighPrecision::test_rbf_like_spd_family`
- `test_spd_matrix_high_precision.py::TestSPDMatrixHighPrecision::test_spd_enclosure_shrin`
- `test_spd_matrix_high_precision.py::TestSPDMatrixHighPrecision::test_two_by_two_spd_logd`

6. Paths, closed-boundary comparisons, domain proofs, subdivision, and automatic control

Challenges uncertain control flow, exact boundary semantics, proof-driven subdivision, and zero-knob policy behavior.

`test_branching_boundary.py` - 3 tests

Checks branch-runtime isolation, architecture boundaries, and internal control-signal behavior. Chosen because path exploration must be local to a certification run and safe under concurrency.

Collected tests:

- `test_branching_boundary.py::TestBranchingBoundary::test_branch_runtime_is_context_local`
- `test_branching_boundary.py::TestBranchingBoundary::test_control_signals_are_not_ordinar`
- `test_branching_boundary.py::TestBranchingBoundary::test_enclosure_branching_layer_does_`

`test_path_aware_branching.py` - 13 tests

Exercises if/elif, strict inequalities, equality boundary paths, nested branches, Boolean short circuiting, nonlinear predicates, branch-local solves, branch budgets, and subdivision composition. Chosen because real engineering code contains regime logic that cannot be replaced by one nominal path.

Collected tests:

- `test_path_aware_branching.py::TestPathAwareBranching::test_adaptive_structural_subdivis`
- `test_path_aware_branching.py::TestPathAwareBranching::test_branch_budget_fails_closed`
- `test_path_aware_branching.py::TestPathAwareBranching::test_branch_local_semantic_solve`
- `test_path_aware_branching.py::TestPathAwareBranching::test_branchless_model_keeps_ordin`
- `test_path_aware_branching.py::TestPathAwareBranching::test_equality_and_inequality_crea`
- `test_path_aware_branching.py::TestPathAwareBranching::test_exact_affine_if_else_is_part`
- `test_path_aware_branching.py::TestPathAwareBranching::test_fixed_subdivision_composes_w`
- `test_path_aware_branching.py::TestPathAwareBranching::test_nested_if_elif_and_downstrea`
- `test_path_aware_branching.py::TestPathAwareBranching::test_nonlinear_multivariate_predi`
- `test_path_aware_branching.py::TestPathAwareBranching::test_path_output_shape_mismatch_i`
- `test_path_aware_branching.py::TestPathAwareBranching::test_piecewise_objects_retain_bra`
- `test_path_aware_branching.py::TestPathAwareBranching::test_short_circuit_and_or_and_tru`

- test_path_aware_branching.py::TestPathAwareBranching::test_strict_inequality_boundary_s

test_domain_proofs.py - 10 tests

Checks physical-domain predicates for sqrt/log/power, proof-driven subdivision, exact boundaries, generator mode, and prepared sessions. Chosen to distinguish a truly invalid model from a valid model whose broad dependency enclosure is temporarily too coarse.

Collected tests:

- test_domain_proofs.py::TestDomainProofs::test_domain_refinement_budget_is_fail_closed
- test_domain_proofs.py::TestDomainProofs::test_domain_refinement_certifies_valid_smooth_
- test_domain_proofs.py::TestDomainProofs::test_generator_representation_uses_same_domain
- test_domain_proofs.py::TestDomainProofs::test_invalid_sqrt_box_still_fails_closed
- test_domain_proofs.py::TestDomainProofs::test_log_remains_strict_at_zero
- test_domain_proofs.py::TestDomainProofs::test_negative_noninteger_power_rejects_zero_bo
- test_domain_proofs.py::TestDomainProofs::test_positive_noninteger_power_accepts_zero_bo
- test_domain_proofs.py::TestDomainProofs::test_prepared_manybox_reuses_domain_architectu
- test_domain_proofs.py::TestDomainProofs::test_sqrt_exact_physical_boundary_without_clip
- test_domain_proofs.py::TestDomainProofs::test_structural_proof_can_avoid_natural_replay

test_configuration_controller.py - 16 tests

Checks zero-knob automatic representation/order selection, retry triggers, intersection portfolios, prepared reuse, and expert overrides. Chosen to prove that automation changes cost/sharpness only and never weakens the baseline certificate.

Collected tests:

- test_configuration_controller.py::TestAutomaticConfigurationController::test_auto_polic
- test_configuration_controller.py::TestAutomaticConfigurationController::test_auto_polic
- test_configuration_controller.py::TestAutomaticConfigurationController::test_auto_polic
- test_configuration_controller.py::TestAutomaticConfigurationController::test_configurat
- test_configuration_controller.py::TestAutomaticConfigurationController::test_explicit_a
- test_configuration_controller.py::TestAutomaticConfigurationController::test_explicit_c
- test_configuration_controller.py::TestAutomaticConfigurationController::test_explicit_r
- test_configuration_controller.py::TestAutomaticConfigurationController::test_failed_opt
- test_configuration_controller.py::TestAutomaticConfigurationController::test_fixed_poli
- test_configuration_controller.py::TestAutomaticConfigurationController::test_policy_val
- test_configuration_controller.py::TestAutomaticConfigurationController::test_prepare_ze
- test_configuration_controller.py::TestAutomaticConfigurationController::test_prepared_a
- test_configuration_controller.py::TestAutomaticConfigurationController::test_reference_
- test_configuration_controller.py::TestAutomaticConfigurationController::test_reference_
- test_configuration_controller.py::TestAutomaticConfigurationController::test_zero_knob_
- test_configuration_controller.py::TestAutomaticConfigurationController::test_zero_knob_

`test_controller_boundary.py` - 3 tests

Checks that controller modules have no forbidden dependencies or old migration state. Chosen to keep policy/orchestration outside the trusted mathematical core.

Collected tests:

- `test_controller_boundary.py::TestControllerBoundary::test_controller_modules_do_not_imp`
- `test_controller_boundary.py::TestControllerBoundary::test_public_api_imports_controller`
- `test_controller_boundary.py::TestControllerBoundary::test_runtime_no_longer_contains_pr`

`test_controller_context.py` - 2 tests

Checks exact restoration of nested controller state and thread-local isolation. Chosen because process-global policy leakage would make results order-dependent and difficult to audit.

Collected tests:

- `test_controller_context.py::TestControllerContext::test_concurrent_threads_do_not_share`
- `test_controller_context.py::TestControllerContext::test_nested_context_restores_parent_`

`test_controller_regression.py` - 4 tests

Bitwise comparisons of coupled/default/subdivision controllers against frozen oracles. Chosen to detect unintended behavioral drift in zero-knob execution.

Collected tests:

- `test_controller_regression.py::TestControllerRegression::test_adaptive_subdivision_matc`
- `test_controller_regression.py::TestControllerRegression::test_default_coupled_controlle`
- `test_controller_regression.py::TestControllerRegression::test_explicit_policy_matches_c`
- `test_controller_regression.py::TestControllerRegression::test_fixed_subdivision_matches`

`test_release_hardening.py` - 10 tests

Release-level checks for environment audit, public behavior, and known edge cases. Chosen to catch packaging/runtime failures that unit tests of individual mathematics would miss.

Collected tests:

- `test_release_hardening.py::TestReleaseHardening::test_branch_boundary_one_ulp_neighborh`
- `test_release_hardening.py::TestReleaseHardening::test_critical_binary64_lattice_scalar_`
- `test_release_hardening.py::TestReleaseHardening::test_domain_boundary_nextafter_neighbo`
- `test_release_hardening.py::TestReleaseHardening::test_environment_exercises_strengthene`
- `test_release_hardening.py::TestReleaseHardening::test_extreme_sqrt_endpoints_satisfy_ex`
- `test_release_hardening.py::TestReleaseHardening::test_near_singular_diagonal_semantic_s`
- `test_release_hardening.py::TestReleaseHardening::test_near_singular_zero_crossing_solve`
- `test_release_hardening.py::TestReleaseHardening::test_reduction_threshold_and_subnormal`

- test_release_hardening.py::TestReleaseHardening::test_semantic_and_generic_diagonal_sol
- test_release_hardening.py::TestReleaseHardening::test_sqrt_and_log_extreme_finite_binar

test_v1_adversarial_ht.py - 2 tests

Freezes the two defects exposed by external heat-exchanger code: strict closed-endpoint comparison pruning and reciprocal proof refinement by physical subdivision. Chosen specifically from adversarial outside-code evidence rather than synthetic convenience.

Collected tests:

- test_v1_adversarial_ht.py::TestV1AdversarialHT::test_reciprocal_proof_failure_triggers_
- test_v1_adversarial_ht.py::TestV1AdversarialHT::test_strict_closed_endpoint_prunes_impo

7. Local semantic dispatch and supported/recognized SciPy semantics

Tests ordinary cross-module scientific Python calls, implicit solves, interpolation, and explicit recognized boundaries.

test_local_semantic_dispatch.py - 9 tests

Checks local preparation of helpers/aliases, no global NumPy mutation, supported boundary behavior, and explicit diagnostics for deferred linear-algebra operations. Chosen so ordinary imported engineering code works without monkey-patching the scientific Python process.

Collected tests:

- test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_active_code_contains_n
- test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_context_entry_itself_d
- test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_direct_imported_solve_
- test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_local_numpy_import_fai
- test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_public_execution_never
- test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_same_module_helper_is_
- test_local_semantic_dispatch.py::TestDeferredLinalgDiagnostics::test_eigvals_on_validat
- test_local_semantic_dispatch.py::TestDeferredLinalgDiagnostics::test_eigvals_proxy_pres
- test_local_semantic_dispatch.py::TestDeferredLinalgDiagnostics::test_imported_eigvals_a

test_implicit_solve_semantics.py - 16 tests

Covers bracketed root_scalar, Colebrook, square fsolve systems, dependency preservation, closures, positional signatures, imported helpers, singular/unprovable cases, and no global SciPy mutation. Chosen because implicit equations are extremely common in engineering and numerical convergence alone is not a proof.

Collected tests:

- test_implicit_solve_semantics.py::test_root_scalar_colebrook_certifies_and_contains_gri
- test_implicit_solve_semantics.py::test_root_scalar_preserves_dependency_downstream
- test_implicit_solve_semantics.py::test_root_scalar_common_positional_signature_is_suppo
- test_implicit_solve_semantics.py::test_validated_root_scalar_without_bracket_fails_expl
- test_implicit_solve_semantics.py::test_root_scalar_unprovable_bracket_is_certification_
- test_implicit_solve_semantics.py::test_fsolve_square_system_certifies_and_contains_grid
- test_implicit_solve_semantics.py::test_fsolve_preserves_dependency_into_residual_and_po
- test_implicit_solve_semantics.py::test_fsolve_full_output_preserves_common_scipy_return
- test_implicit_solve_semantics.py::test_fsolve_singular_case_never_returns_false_certifi
- test_implicit_solve_semantics.py::test_root_scalar_nested_closure_parameters_are_suppor
- test_implicit_solve_semantics.py::test_fsolve_nested_closure_parameters_are_supported
- test_implicit_solve_semantics.py::test_fsolve_elementary_nonlinearity_certifies
- test_implicit_solve_semantics.py::test_imported_pure_python_root_callback_is_prepared_l
- test_implicit_solve_semantics.py::test_scipy_optimize_functions_are_not_globally_mutate
- test_implicit_solve_semantics.py::test_imported_pure_python_module_helper_with_root_sca
- test_implicit_solve_semantics.py::test_imported_pure_python_module_helper_with_fsolve_i

test_regular_grid_interpolator_semantics.py - 13 tests

Covers validated linear RegularGridInterpolator inside cells, across cell boundaries, descending grids, vector outputs, fill/extrapolation behavior, unsupported methods, and cross-module use. Chosen because tabulated maps are pervasive in engineering and piecewise multilinear interpolation is a natural V1 semantic.

Collected tests:

- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_single_cell
- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_crosses_gri
- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_constructor
- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_descending_
- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_linear_extr
- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_finite_fill
- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_bounds_erro
- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_non_linear_
- test_regular_grid_interpolator_semantics.py::test_scipy_linalg_eigh_fails_with_explicit
- test_regular_grid_interpolator_semantics.py::test_scipy_linalg_eigh_numeric_calls_remai
- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_imported_he
- test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_uncertain_t
- test_regular_grid_interpolator_semantics.py::test_scipy_module_eigh_path_fails_explicit

test_scipy_compatibility_boundary.py - 3 tests

Checks recognized SciPy compatibility boundaries and explicit unsupported-operation messages, including eigh. Chosen so unsupported common operations fail intelligibly rather than through object-array/type errors.

Collected tests:

- test_scipy_compatibility_boundary.py::TestScipyCompatibilityBoundary::test_scipy_dispat
- test_scipy_compatibility_boundary.py::TestScipyCompatibilityBoundary::test_scipy_is_not
- test_scipy_compatibility_boundary.py::TestScipyCompatibilityBoundary::test_top_level_sc

test_semantic_boundary.py - 2 tests

Checks dependency direction and isolation of semantic execution machinery. Chosen to prevent dispatcher conveniences from contaminating trusted numerical layers.

Collected tests:

- test_semantic_boundary.py::TestSemanticBoundary::test_semantic_module_owns_solve_contex
- test_semantic_boundary.py::TestSemanticBoundary::test_semantic_solve_module_does_not_im

8. Public API, input specification, many-box/accelerator execution, application theorems, package boundaries, and typed failures

Validates user-facing contracts, exact-real physical-input specification, repeated and parallel certification workflows, application extension mechanisms, trust labels for heterogeneous acceleration, architectural separation, and fail-closed semantics.

test_prepared_manybox.py - 10 tests

Checks prepared sessions, caching, rigorous restriction to interior subboxes, generator-ID preservation, batch/thread execution, process limitations, compact and streaming interfaces. Chosen for workflows that certify many related boxes efficiently without re-running the model unnecessarily.

Collected tests:

- test_prepared_manybox.py::TestPreparedSession::test_boundary_query_falls_back_when_symm
- test_prepared_manybox.py::TestPreparedSession::test_certify_many_endpoint_array_and_thr
- test_prepared_manybox.py::TestPreparedSession::test_compact_many_box_bounds_interfaces
- test_prepared_manybox.py::TestPreparedSession::test_exact_cache_avoids_model_reexecutio
- test_prepared_manybox.py::TestPreparedSession::test_exact_cache_metadata_does_not_mutat
- test_prepared_manybox.py::TestPreparedSession::test_fresh_prepared_execution_matches_on
- test_prepared_manybox.py::TestPreparedSession::test_generator_restriction_preserves_sha
- test_prepared_manybox.py::TestPreparedSession::test_interior_subbox_restricts_certifica
- test_prepared_manybox.py::TestPreparedSession::test_process_mode_rejects_unpickleable_l
- test_prepared_manybox.py::TestPreparedSession::test_streaming_many_box_interface

test_parallel_accelerators.py - 10 tests

Checks the V1 heterogeneous-execution contract: automatic worker/backend resolution preserves serial certificates; top-level and prepared many-box APIs

agree; execution metadata explicitly records the CPU proof backend; local closures can use process workers through optional `cloudpickle`; accelerator discovery never labels MPS/CUDA as trusted proof; Torch proposal batching is explicitly non-rigorous; and an unavailable requested MPS device fails clearly. Chosen because parallel throughput is useful only if it cannot blur the boundary between a fast proposal and the independently verified CPU certificate.

Collected tests:

- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_accelerator_report_nev`
- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_cpu_torch_proposal_map`
- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_explicit_unavailable_m`
- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_manybox_execution_meta`
- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_process_parallel_local`
- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_top_level_enclose_many`
- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_top_level_manybox_retu`
- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_workers_auto_and_paral`
- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_safe_auto_small_batch_`
- `test_parallel_accelerators.py::TestParallelAndAccelerators::test_auto_proposal_device_i`

`test_application_theorems.py` - 8 tests

Checks the public theorem context, certified arithmetic, assumptions, interval intersection, upper/lower portfolios, metadata, and box inputs. Chosen to support application-specific rigorous mathematics while maintaining the permanent application -> AutoIA dependency direction.

Collected tests:

- `test_application_theorems.py::ApplicationTheoremPublicAPITests::test_certified_interval`
- `test_application_theorems.py::ApplicationTheoremPublicAPITests::test_certified_scalar_a`
- `test_application_theorems.py::ApplicationTheoremPublicAPITests::test_checked_assumption`
- `test_application_theorems.py::ApplicationTheoremPublicAPITests::test_context_enclose_hi`
- `test_application_theorems.py::ApplicationTheoremPublicAPITests::test_lower_bound_portfo`
- `test_application_theorems.py::ApplicationTheoremPublicAPITests::test_public_arithmetic_`
- `test_application_theorems.py::ApplicationTheoremPublicAPITests::test_public_interval_bo`
- `test_application_theorems.py::ApplicationTheoremPublicAPITests::test_theorem_metadata_a`

`test_application_package_boundary.py` - 2 tests

Checks that AutoIA source and install metadata contain no ExactBO/GP application modules. Chosen to enforce the architectural separation discovered during the ExactBO work.

Collected tests:

- `test_application_package_boundary.py::ApplicationPackageBoundaryTests::test_autoia_inst`
- `test_application_package_boundary.py::ApplicationPackageBoundaryTests::test_autoia_pack`

`test_input_specification.py` - 17 tests

Checks the V1.0.1 physical-input specification boundary. Exact decimal/rational endpoint requests must be outwardly enclosed by finite binary64 proof endpoints and verified using exact rational comparison; ordinary floats are rejected by the exact-real constructor because their pre-rounding provenance is unavailable. The same file checks explicitly labeled one-ulp padding, result provenance, and prepared-cache metadata. Chosen because a machine-rigorous proof over the wrong input set is not the theorem the user intended.

Collected tests:

- `test_input_specification.py::TestExactRealInputConstruction::test_binary64_representabl`
- `test_input_specification.py::TestExactRealInputConstruction::test_direct_constructor_ke`
- `test_input_specification.py::TestExactRealInputConstruction::test_exact_decimal_interva`
- `test_input_specification.py::TestExactRealInputConstruction::test_exact_endpoint_order_`
- `test_input_specification.py::TestExactRealInputConstruction::test_float_arguments_are_r`
- `test_input_specification.py::TestExactRealInputConstruction::test_fraction_and_decimal_`
- `test_input_specification.py::TestExactRealInputConstruction::test_interval_metadata_sur`
- `test_input_specification.py::TestExactRealInputConstruction::test_negative_nonrepresent`
- `test_input_specification.py::TestExactRealInputConstruction::test_nonfinite_and_out_of_`
- `test_input_specification.py::TestExactRealInputConstruction::test_nonrepresentable_exac`
- `test_input_specification.py::TestExactRealInputConstruction::test_scientific_decimal_st`
- `test_input_specification.py::TestExactRealInputConstruction::test_subnormal_neighborhoo`
- `test_input_specification.py::TestOutwardPadding::test_one_ulp_outward_padding`
- `test_input_specification.py::TestOutwardPadding::test_padding_max_finite_fails_closed_b`
- `test_input_specification.py::TestOutwardPadding::test_padding_point_zero_expands_both_s`
- `test_input_specification.py::TestResultInputMetadata::test_prepared_cache_metadata_foll`
- `test_input_specification.py::TestResultInputMetadata::test_primary_result_records_exact`

`test_public_contract.py` - 3 tests

Small end-to-end contract tests for analytic containment, ordinary cases, and zero-crossing failure. Chosen as stable user-facing smoke tests of the package promise.

Collected tests:

- `test_public_contract.py::TestPublicContract::test_manual_2x2_needs_no_semantic_certific`
- `test_public_contract.py::TestPublicContract::test_spring_contains_analytic_range`
- `test_public_contract.py::TestPublicContract::test_zero_crossing_denominator_fails_close`

`test_typed_failures.py` - 12 tests

Checks that unsupported operations, invalid domains, backend/environment failures, user model exceptions, uncertain exponents, and zero-crossing division receive the correct public failure type. Chosen because fail-closed behavior is part of the V1 correctness contract.

Collected tests:

- `test_typed_failures.py::TestTypedFailures::test_all_public_execution_failures_share_bas`
- `test_typed_failures.py::TestTypedFailures::test_bad_representation_is_invalid_input`
- `test_typed_failures.py::TestTypedFailures::test_empty_input_box_is_invalid_input`
- `test_typed_failures.py::TestTypedFailures::test_interval_semantic_solve_contraction_fai`
- `test_typed_failures.py::TestTypedFailures::test_negative_power_zero_crossing_is_certifi`
- `test_typed_failures.py::TestTypedFailures::test_policy_validation_is_typed`
- `test_typed_failures.py::TestTypedFailures::test_public_backend_environment_failure_is_t`
- `test_typed_failures.py::TestTypedFailures::test_public_interval_validation_is_typed`
- `test_typed_failures.py::TestTypedFailures::test_uncertain_exponent_is_unsupported`
- `test_typed_failures.py::TestTypedFailures::test_unsupported_transcendental_is_typed`
- `test_typed_failures.py::TestTypedFailures::test_user_model_failure_is_not_mislabeled_as`
- `test_typed_failures.py::TestTypedFailures::test_zero_crossing_division_is_certification`

`test_migration_retirement.py` - 3 tests

Checks that old migration/oracle modules are absent from production and survive only in test support. Chosen to ensure the current architecture is genuinely the shipped architecture rather than a compatibility facade over retired code.

Collected tests:

- `test_migration_retirement.py::TestMigrationRetirement::test_historical_oracle_lives_onl`
- `test_migration_retirement.py::TestMigrationRetirement::test_migration_modules_are_absen`
- `test_migration_retirement.py::TestMigrationRetirement::test_production_modules_do_not_i`

`test_reference_regression.py` - 1 tests

Keeps a small frozen end-to-end reference result. Chosen as a simple sentinel for broad behavioral drift across many internal changes.

Collected tests:

- `test_reference_regression.py::TestReferenceRegression::test_coupled_semantic_binary64_r`

9. External and independently authored engineering validation

Challenges the system with code/data patterns not designed to flatter or simplify AutoIA.

`test_engineering_validation_corpus.py` - 5 tests

Checks a curated engineering corpus for source independence from AutoIA, nominal execution, real singularity witnesses, and representative rigorous certification. Chosen to prevent the package from being validated only on functions written specifically for AutoIA.

Collected tests:

- `test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_case_names`
- `test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_every_mode`
- `test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_generator_`
- `test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_lmtd_inval`
- `test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_model_sour`

`test_external_engineering_validation.py` - 5 tests

Checks vendored independently authored engineering source cases, nominal executability, representative certification, and real branch crossing. Chosen as stronger evidence that ordinary external engineering code can traverse AutoIA semantics without being rewritten for the tool.

Collected tests:

- `test_external_engineering_validation.py::TestExternalEngineeringValidation::test_every_`
- `test_external_engineering_validation.py::TestExternalEngineeringValidation::test_extern`
- `test_external_engineering_validation.py::TestExternalEngineeringValidation::test_extern`
- `test_external_engineering_validation.py::TestExternalEngineeringValidation::test_repres`
- `test_external_engineering_validation.py::TestExternalEngineeringValidation::test_vendor`

Cross-cutting evidence that is intentionally outside the 381 count

Numbered examples

The 27 numbered examples are executable demonstrations, not unit-test count inflation. They cover the zero-knob API, scalar/vector nonlinear functions, linear solves, branching, subdivision, prepared sessions, shared-generator cancellation, normal functions, failure handling, application-theorem composition, implicit solves, cross-module helpers, regular-grid interpolation, explicit unsupported `eigh`, expert representations/ranging, closed-boundary/domain refinement, structured SPD matrix families, multicore CPU execution, compact/streaming many-box workflows, accelerator trust labeling, and shrinking RBF-like SPD families.

ExactBO compatibility

ExactBO is intentionally a separate application package. Its compatibility suite verifies that changes to AutoIA do not pull GP/Bayesian-optimization theory back into the core or break the public application interface. Against the final clean dev14 wheel: **43 tests passed and 1 optional live-Torch/BoTorch-dependent test was skipped**.

External/high-precision audit scripts

The repository also retains audit tools and historical validation logs for exact-rational reductions, elementary functions, domain proofs, general solves, packed

Bernstein/monotonicity, engineering corpora, and previous release checkpoints. These provide additional numerical witnesses but are not double-counted in the 381 stable pytest cases.

What the suite does not prove

Passing 381 tests is strong engineering evidence, not a formal proof that every possible Python program is certified correctly. The mathematical soundness claim still comes from the documented proof rules and trusted arithmetic assumptions. The test suite instead attacks likely implementation failure modes and checks that the implementation conforms to those rules over a broad and adversarial set of cases.

In particular, the suite does not claim V1 semantics for every SciPy function. Operations such as `scipy.linalg.eigh` are deliberately recognized and rejected with an explicit unsupported-operation diagnostic until theorem-backed semantics are implemented.

The suite also does not make sharpness a universal theorem. Some large boxes can be rigorous but loose. Shrinking-box studies, subdivision tests, and outside engineering examples are used to check that supported semantics improve sensibly under localization and that the controller does not confuse proof difficulty with physical invalidity.

Why the 381-test design is appropriate for a V1 release

A certification library needs a different validation balance from an ordinary numerical library. The highest priority is false-positive prevention: no test should reward AutoIA for returning a narrow interval if the proof obligation was not actually established. That is why the suite devotes substantial weight to trusted arithmetic, exact boundaries, typed failures, domain proofs, path semantics, and architectural separation.

At the same time, a mathematically pure core that cannot execute ordinary engineering code would not meet the project goal. The array, local-dispatch, implicit-solve, interpolation, external-corpus, and adversarial tests therefore challenge usability without changing the proof contract.

The resulting evidence base intentionally spans **proof correctness, representation correctness, program semantics, failure correctness, architecture, performance-preserving rigor, and external engineering realism**. That breadth is the reason the suite is more meaningful than 381 variations of a single enclosure formula.

Appendix A. Complete named-test inventory

1. `tests/test_application_package_boundary.py::ApplicationPackageBoundaryTests::test_autoi`
2. `tests/test_application_package_boundary.py::ApplicationPackageBoundaryTests::test_autoi`

3. tests/test_application_theorems.py::ApplicationTheoremPublicAPITests::test_certified_in
4. tests/test_application_theorems.py::ApplicationTheoremPublicAPITests::test_certified_sc
5. tests/test_application_theorems.py::ApplicationTheoremPublicAPITests::test_checked_assu
6. tests/test_application_theorems.py::ApplicationTheoremPublicAPITests::test_context_encl
7. tests/test_application_theorems.py::ApplicationTheoremPublicAPITests::test_lower_bound_
8. tests/test_application_theorems.py::ApplicationTheoremPublicAPITests::test_public_arith
9. tests/test_application_theorems.py::ApplicationTheoremPublicAPITests::test_public_inter
10. tests/test_application_theorems.py::ApplicationTheoremPublicAPITests::test_theorem_meta
11. tests/test_array_semantics.py::TestArrayConstructionAndMutation::test_array_constructor
12. tests/test_array_semantics.py::TestArrayConstructionAndMutation::test_direct_zeros_alia
13. tests/test_array_semantics.py::TestArrayConstructionAndMutation::test_zero_initialized_
14. tests/test_array_semantics.py::TestArrayConstructionAndMutation::test_zeros_matrix_asse
15. tests/test_array_semantics.py::TestBroadcastAndStructuralArraySemantics::test_preexisti
16. tests/test_array_semantics.py::TestBroadcastAndStructuralArraySemantics::test_stack_con
17. tests/test_array_semantics.py::TestBroadcastAndStructuralArraySemantics::test_two_dimen
18. tests/test_array_semantics.py::TestReductionsAndLinearAlgebra::test_direct_dot_alias
19. tests/test_array_semantics.py::TestReductionsAndLinearAlgebra::test_direct_sum_alias
20. tests/test_array_semantics.py::TestReductionsAndLinearAlgebra::test_dot_matmul_inner_ou
21. tests/test_array_semantics.py::TestReductionsAndLinearAlgebra::test_sum_mean_prod_axes_
22. tests/test_array_semantics.py::TestNormsAndResultShape::test_matrix_frobenius_norm
23. tests/test_array_semantics.py::TestNormsAndResultShape::test_matrix_spectral_norm_is_ex
24. tests/test_array_semantics.py::TestNormsAndResultShape::test_scalar_output_shape_metada
25. tests/test_array_semantics.py::TestNormsAndResultShape::test_vector_norm_and_alias
26. tests/test_array_semantics.py::TestNormsAndResultShape::test_vector_norm_crossing_zero_
27. tests/test_batched_triangular_interval.py::TestBatchedTriangularIntervalSolve::test_arb
28. tests/test_batched_triangular_interval.py::TestBatchedTriangularIntervalSolve::test_poi
29. tests/test_batched_triangular_interval.py::TestBatchedTriangularIntervalSolve::test_ran
30. tests/test_batched_triangular_interval.py::TestBatchedTriangularIntervalSolve::test_rej
31. tests/test_branching_boundary.py::TestBranchingBoundary::test_branch_runtime_is_context
32. tests/test_branching_boundary.py::TestBranchingBoundary::test_control_signals_are_not_c
33. tests/test_branching_boundary.py::TestBranchingBoundary::test_enclosure_branching_layer
34. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_auto
35. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_auto
36. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_auto
37. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_conf
38. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_expl
39. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_expl
40. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_expl
41. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_fail
42. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_fixe
43. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_poli
44. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_prep
45. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_prep
46. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_refe
47. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_refe
48. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_zero

49. tests/test_configuration_controller.py::TestAutomaticConfigurationController::test_zero
50. tests/test_controller_boundary.py::TestControllerBoundary::test_controller_modules_do_n
51. tests/test_controller_boundary.py::TestControllerBoundary::test_public_api_imports_cont
52. tests/test_controller_boundary.py::TestControllerBoundary::test_runtime_no_longer_conta
53. tests/test_controller_context.py::TestControllerContext::test_concurrent_threads_do_not
54. tests/test_controller_context.py::TestControllerContext::test_nested_context_restores_p
55. tests/test_controller_regression.py::TestControllerRegression::test_adaptive_subdivisio
56. tests/test_controller_regression.py::TestControllerRegression::test_default_coupled_con
57. tests/test_controller_regression.py::TestControllerRegression::test_explicit_policy_mat
58. tests/test_controller_regression.py::TestControllerRegression::test_fixed_subdivision_m
59. tests/test_domain_proofs.py::TestDomainProofs::test_domain_refinement_budget_is_fail_cl
60. tests/test_domain_proofs.py::TestDomainProofs::test_domain_refinement_certifies_valid_s
61. tests/test_domain_proofs.py::TestDomainProofs::test_generator_representation_uses_same_
62. tests/test_domain_proofs.py::TestDomainProofs::test_invalid_sqrt_box_still_fails_closed
63. tests/test_domain_proofs.py::TestDomainProofs::test_log_remains_strict_at_zero
64. tests/test_domain_proofs.py::TestDomainProofs::test_negative_noninteger_power_rejects_z
65. tests/test_domain_proofs.py::TestDomainProofs::test_positive_noninteger_power_accepts_z
66. tests/test_domain_proofs.py::TestDomainProofs::test_prepared_manybox_reuses_domain_arch
67. tests/test_domain_proofs.py::TestDomainProofs::test_sqrt_exact_physical_boundary_withou
68. tests/test_domain_proofs.py::TestDomainProofs::test_structural_proof_can_avoid_natural_
69. tests/test_elementary_boundary.py::TestElementaryBoundary::test_elementary_core_depends
70. tests/test_elementary_boundary.py::TestElementaryBoundary::test_enclosure_elementary_do
71. tests/test_elementary_execution.py::TestElementaryExecution::test_all_five_polynomial_f
72. tests/test_elementary_execution.py::TestElementaryExecution::test_composed_exp_log_cont
73. tests/test_elementary_execution.py::TestElementaryExecution::test_direct_numpy_alias_is
74. tests/test_elementary_execution.py::TestElementaryExecution::test_elementary_dispatch_d
75. tests/test_elementary_execution.py::TestElementaryExecution::test_elementary_postproces
76. tests/test_elementary_execution.py::TestElementaryExecution::test_interval_representati
77. tests/test_elementary_execution.py::TestElementaryExecution::test_log10_domain_failure_
78. tests/test_elementary_execution.py::TestElementaryExecution::test_log10_module_numpy_an
79. tests/test_elementary_execution.py::TestElementaryExecution::test_math_module_and_direc
80. tests/test_elementary_execution.py::TestElementaryExecution::test_multivariate_nonlinea
81. tests/test_elementary_execution.py::TestElementaryExecution::test_object_array_ufunc_di
82. tests/test_elementary_execution.py::TestElementaryExecution::test_polynomial_boundary_d
83. tests/test_elementary_execution.py::TestElementaryExecution::test_real_domain_failures_
84. tests/test_elementary_execution.py::TestElementaryExecution::test_sqrt_touching_zero_is
85. tests/test_enclosure_boundary.py::TestEnclosureBoundary::test_enclosure_modules_do_not_
86. tests/test_enclosure_boundary.py::TestEnclosureBoundary::test_public_representations_fa
87. tests/test_enclosure_boundary.py::TestEnclosureBoundary::test_validated_value_has_no_co
88. tests/test_enclosure_regression.py::TestEnclosureRegression::test_interval_enclosure_ma
89. tests/test_enclosure_regression.py::TestEnclosureRegression::test_mixed_interval_polync
90. tests/test_enclosure_regression.py::TestEnclosureRegression::test_parameter_and_generic
91. tests/test_enclosure_regression.py::TestEnclosureRegression::test_parameterized_alias_i
92. tests/test_enclosure_regression.py::TestEnclosureRegression::test_piecewise_hull_matche
93. tests/test_enclosure_regression.py::TestEnclosureRegression::test_reciprocal_and_divisi
94. tests/test_enclosure_regression.py::TestEnclosureRegression::test_validated_value_inter

95. tests/test_enclosure_regression.py::TestEnclosureRegression::test_validated_value_polyn
96. tests/test_enclosure_regression.py::TestRepresentationContract::test_mixed_representati
97. tests/test_enclosure_regression.py::TestRepresentationContract::test_validated_value_ac
98. tests/test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_case
99. tests/test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_ever
100. tests/test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_gene
101. tests/test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_lmtd
102. tests/test_engineering_validation_corpus.py::TestEngineeringValidationCorpus::test_mode
103. tests/test_external_engineering_validation.py::TestExternalEngineeringValidation::test_
104. tests/test_external_engineering_validation.py::TestExternalEngineeringValidation::test_
105. tests/test_external_engineering_validation.py::TestExternalEngineeringValidation::test_
106. tests/test_external_engineering_validation.py::TestExternalEngineeringValidation::test_
107. tests/test_external_engineering_validation.py::TestExternalEngineeringValidation::test_
108. tests/test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_direct_auxil
109. tests/test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_generator_in
110. tests/test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_multiple_rhs
111. tests/test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_remaining_bo
112. tests/test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_rhs_generato
113. tests/test_generator_aware_solve.py::TestGeneratorAwareSemanticSolve::test_shared_rhs_g
114. tests/test_generator_boundary.py::TestGeneratorBoundary::test_generator_layer_does_not_
115. tests/test_generator_boundary.py::TestGeneratorBoundary::test_generator_representation_
116. tests/test_implicit_solve_semantics.py::test_root_scalar_colebrook_certifies_and_contai
117. tests/test_implicit_solve_semantics.py::test_root_scalar_preserves_dependency_downstrea
118. tests/test_implicit_solve_semantics.py::test_root_scalar_common_positional_signature_is
119. tests/test_implicit_solve_semantics.py::test_validated_root_scalar_without_bracket_fail
120. tests/test_implicit_solve_semantics.py::test_root_scalar_unprovable_bracket_is_certific
121. tests/test_implicit_solve_semantics.py::test_fsolve_square_system_certifies_and_contain
122. tests/test_implicit_solve_semantics.py::test_fsolve_preserves_dependency_into_residual_
123. tests/test_implicit_solve_semantics.py::test_fsolve_full_output_preserves_common_scipy_
124. tests/test_implicit_solve_semantics.py::test_fsolve_singular_case_never_returns_false_c
125. tests/test_implicit_solve_semantics.py::test_root_scalar_nested_closure_parameters_are_
126. tests/test_implicit_solve_semantics.py::test_fsolve_nested_closure_parameters_are_suppo
127. tests/test_implicit_solve_semantics.py::test_fsolve_elementary_nonlinearity_certifies
128. tests/test_implicit_solve_semantics.py::test_imported_pure_python_root_callback_is_prep
129. tests/test_implicit_solve_semantics.py::test_scipy_optimize_functions_are_not_globally_
130. tests/test_implicit_solve_semantics.py::test_imported_pure_python_module_helper_with_ro
131. tests/test_implicit_solve_semantics.py::test_imported_pure_python_module_helper_with_fs
132. tests/test_input_specification.py::TestExactRealInputConstruction::test_binary64_repres
133. tests/test_input_specification.py::TestExactRealInputConstruction::test_direct_construc
134. tests/test_input_specification.py::TestExactRealInputConstruction::test_exact_decimal_i
135. tests/test_input_specification.py::TestExactRealInputConstruction::test_exact_endpoint_
136. tests/test_input_specification.py::TestExactRealInputConstruction::test_float_arguments
137. tests/test_input_specification.py::TestExactRealInputConstruction::test_fraction_and_de
138. tests/test_input_specification.py::TestExactRealInputConstruction::test_interval_metada
139. tests/test_input_specification.py::TestExactRealInputConstruction::test_negative_nonrep
140. tests/test_input_specification.py::TestExactRealInputConstruction::test_nonfinite_and_c

141. tests/test_input_specification.py::TestExactRealInputConstruction::test_nonrepresentabl
142. tests/test_input_specification.py::TestExactRealInputConstruction::test_scientific_deci
143. tests/test_input_specification.py::TestExactRealInputConstruction::test_subnormal_neigh
144. tests/test_input_specification.py::TestOutwardPadding::test_one_ulp_outward_padding
145. tests/test_input_specification.py::TestOutwardPadding::test_padding_max_finite_fails_cl
146. tests/test_input_specification.py::TestOutwardPadding::test_padding_point_zero_expands_
147. tests/test_input_specification.py::TestResultInputMetadata::test_prepared_cache_metadat
148. tests/test_input_specification.py::TestResultInputMetadata::test_primary_result_records
149. tests/test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_au
150. tests/test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_ba
151. tests/test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_ba
152. tests/test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_in
153. tests/test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_lo
154. tests/test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_lo
155. tests/test_interval_reduction_performance.py::TestVectorizedIntervalReductions::test_ma
156. tests/test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_active_code_cont
157. tests/test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_context_entry_it
158. tests/test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_direct_imported_
159. tests/test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_local_numpy_impo
160. tests/test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_public_execution
161. tests/test_local_semantic_dispatch.py::TestLocalSemanticDispatch::test_same_module_help
162. tests/test_local_semantic_dispatch.py::TestDeferredLinalgDiagnostics::test_eigvals_on_v
163. tests/test_local_semantic_dispatch.py::TestDeferredLinalgDiagnostics::test_eigvals_prox
164. tests/test_local_semantic_dispatch.py::TestDeferredLinalgDiagnostics::test_imported_eig
165. tests/test_migration_retirement.py::TestMigrationRetirement::test_historical_oracle_liv
166. tests/test_migration_retirement.py::TestMigrationRetirement::test_migration_modules_are
167. tests/test_migration_retirement.py::TestMigrationRetirement::test_production_modules_do
168. tests/test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_dense_st
169. tests/test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_family_b
170. tests/test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_fixed_di
171. tests/test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_full_coe
172. tests/test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_full_fam
173. tests/test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_one_dime
174. tests/test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_portfoli
175. tests/test_packed_polynomial_bernstein.py::TestPackedPolynomialBernstein::test_reduced_
176. tests/test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_ba
177. tests/test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_de
178. tests/test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_ha
179. tests/test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_mo
180. tests/test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_pa
181. tests/test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_po
182. tests/test_packed_polynomial_monotonicity.py::TestPackedPolynomialMonotonicity::test_se
183. tests/test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_family_axis_
184. tests/test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_interval_fam
185. tests/test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_interval_fam
186. tests/test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_interval_min

187. tests/test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_plan_is_canc
188. tests/test_packed_polynomial_ranging.py::TestPackedPolynomialRanging::test_point_family
189. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_accelerator_repo
190. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_auto_proposal_de
191. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_cpu_torch_propos
192. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_explicit_unavail
193. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_manybox_executio
194. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_process_parallel
195. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_safe_auto_small_
196. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_top_level_enclos
197. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_top_level_manybo
198. tests/test_parallel_accelerators.py::TestParallelAndAccelerators::test_workers_auto_and
199. tests/test_path_aware_branching.py::TestPathAwareBranching::test_adaptive_structural_su
200. tests/test_path_aware_branching.py::TestPathAwareBranching::test_branch_budget_fails_cl
201. tests/test_path_aware_branching.py::TestPathAwareBranching::test_branch_local_semantic_
202. tests/test_path_aware_branching.py::TestPathAwareBranching::test_branchless_model_keeps
203. tests/test_path_aware_branching.py::TestPathAwareBranching::test_equality_and_inequalit
204. tests/test_path_aware_branching.py::TestPathAwareBranching::test_exact_affine_if_else_i
205. tests/test_path_aware_branching.py::TestPathAwareBranching::test_fixed_subdivision_comp
206. tests/test_path_aware_branching.py::TestPathAwareBranching::test_nested_if_elif_and_dow
207. tests/test_path_aware_branching.py::TestPathAwareBranching::test_nonlinear_multivariate
208. tests/test_path_aware_branching.py::TestPathAwareBranching::test_path_output_shape_mism
209. tests/test_path_aware_branching.py::TestPathAwareBranching::test_piecewise_objects_reta
210. tests/test_path_aware_branching.py::TestPathAwareBranching::test_short_circuit_and_or_a
211. tests/test_path_aware_branching.py::TestPathAwareBranching::test_strict_inequality_boun
212. tests/test_performance_kernels.py::TestVectorizedTrustedReductions::test_auto_mode_keep
213. tests/test_performance_kernels.py::TestVectorizedTrustedReductions::test_public_backend
214. tests/test_performance_kernels.py::TestVectorizedTrustedReductions::test_vectorized_dot
215. tests/test_performance_kernels.py::TestVectorizedTrustedReductions::test_vectorized_non
216. tests/test_performance_kernels.py::TestBatchedProposalFactorization::test_polynomial_pr
217. tests/test_polynomial_boundary.py::TestPolynomialBoundary::test_polynomial_modules_do_n
218. tests/test_polynomial_boundary.py::TestPolynomialBoundary::test_ranging_depends_on_alge
219. tests/test_polynomial_regression.py::TestPolynomialRegression::test_degree_and_reduced_
220. tests/test_polynomial_regression.py::TestPolynomialRegression::test_derivative_and_monc
221. tests/test_polynomial_regression.py::TestPolynomialRegression::test_monomial_bounds_are
222. tests/test_polynomial_regression.py::TestPolynomialRegression::test_multiindex_and_clea
223. tests/test_polynomial_regression.py::TestPolynomialRegression::test_polynomial_addition
224. tests/test_polynomial_regression.py::TestPolynomialRegression::test_primary_range_metho
225. tests/test_polynomial_regression.py::TestPolynomialRegression::test_reciprocal_predicto
226. tests/test_polynomial_regression.py::TestPolynomialRegression::test_reduced_bernstein_i
227. tests/test_power_execution.py::TestPowerExecution::test_domain_failures
228. tests/test_power_execution.py::TestPowerExecution::test_fractional_positive_and_negativ
229. tests/test_power_execution.py::TestPowerExecution::test_half_power_reuses_sqrt_quality
230. tests/test_power_execution.py::TestPowerExecution::test_interval_even_power_preserves_s
231. tests/test_power_execution.py::TestPowerExecution::test_math_pow_and_alias
232. tests/test_power_execution.py::TestPowerExecution::test_negative_integer_powers

233. tests/test_power_execution.py::TestPowerExecution::test_numpy_power_and_alias
234. tests/test_power_execution.py::TestPowerExecution::test_power_dispatch_does_not_mutate_
235. tests/test_power_execution.py::TestPowerExecution::test_uncertain_exponent_unsupported
236. tests/test_prepared_manybox.py::TestPreparedSession::test_boundary_query_falls_back_whe
237. tests/test_prepared_manybox.py::TestPreparedSession::test_certify_many_endpoint_array_a
238. tests/test_prepared_manybox.py::TestPreparedSession::test_compact_many_box_bounds_inter
239. tests/test_prepared_manybox.py::TestPreparedSession::test_exact_cache_avoids_model_reex
240. tests/test_prepared_manybox.py::TestPreparedSession::test_exact_cache_metadata_does_not
241. tests/test_prepared_manybox.py::TestPreparedSession::test_fresh_prepared_execution_matc
242. tests/test_prepared_manybox.py::TestPreparedSession::test_generator_restriction_preserv
243. tests/test_prepared_manybox.py::TestPreparedSession::test_interior_subbox_restricts_cer
244. tests/test_prepared_manybox.py::TestPreparedSession::test_process_mode_supports_local_m
245. tests/test_prepared_manybox.py::TestPreparedSession::test_streaming_many_box_interface
246. tests/test_public_contract.py::TestPublicContract::test_manual_2x2_needs_no_semantic_ce
247. tests/test_public_contract.py::TestPublicContract::test_spring_contains_analytic_range
248. tests/test_public_contract.py::TestPublicContract::test_zero_crossing_denominator_fails
249. tests/test_reductions_exact.py::TestExactReductions::test_adversarial_cancellation_dot_
250. tests/test_reductions_exact.py::TestExactReductions::test_certificate_matrix_helpers_ag
251. tests/test_reductions_exact.py::TestExactReductions::test_interval_matvec_against_exact
252. tests/test_reductions_exact.py::TestExactReductions::test_nonnegative_upper_reductions_
253. tests/test_reductions_exact.py::TestExactReductions::test_point_matrix_products_against
254. tests/test_reductions_exact.py::TestExactReductions::test_point_times_interval_reductio
255. tests/test_reductions_exact.py::TestExactReductions::test_random_dot_products_against_e
256. tests/test_reference_regression.py::TestReferenceRegression::test_coupled_semantic_bina
257. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_singl
258. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_cross
259. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_const
260. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_desce
261. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_linea
262. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_finit
263. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_bound
264. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_non_l
265. tests/test_regular_grid_interpolator_semantics.py::test_scipy_linalg_eigh_fails_with_ex
266. tests/test_regular_grid_interpolator_semantics.py::test_scipy_linalg_eigh_numeric_calls
267. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_impor
268. tests/test_regular_grid_interpolator_semantics.py::test_regular_grid_interpolator_uncer
269. tests/test_regular_grid_interpolator_semantics.py::test_scipy_module_eigh_path_fails_ex
270. tests/test_release_hardening.py::TestReleaseHardening::test_branch_boundary_one_ulp_nei
271. tests/test_release_hardening.py::TestReleaseHardening::test_critical_binary64_lattice_s
272. tests/test_release_hardening.py::TestReleaseHardening::test_domain_boundary_nextafter_n
273. tests/test_release_hardening.py::TestReleaseHardening::test_environment_exercises_stren
274. tests/test_release_hardening.py::TestReleaseHardening::test_extreme_sqrt_endpoints_sati
275. tests/test_release_hardening.py::TestReleaseHardening::test_near_singular_diagonal_sema
276. tests/test_release_hardening.py::TestReleaseHardening::test_near_singular_zero_crossing
277. tests/test_release_hardening.py::TestReleaseHardening::test_reduction_threshold_and_sub
278. tests/test_release_hardening.py::TestReleaseHardening::test_semantic_and_generic_diagon

279. tests/test_release_hardening.py::TestReleaseHardening::test_sqrt_and_log_extreme_finite
280. tests/test_scalar_binary64_exact.py::TestBinary64Environment::test_environment_audit_pa
281. tests/test_scalar_binary64_exact.py::TestBinary64Environment::test_integer_interval_con
282. tests/test_scalar_binary64_exact.py::TestBinary64Environment::test_trusted_interval_can
283. tests/test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_division_by_int
284. tests/test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_edge_case_point
285. tests/test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_random_interval
286. tests/test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_random_raw_bina
287. tests/test_scalar_binary64_exact.py::TestScalarExactRationalAudit::test_signed_zero_and
288. tests/test_scipy_compatibility_boundary.py::TestScipyCompatibilityBoundary::test_scipy_
289. tests/test_scipy_compatibility_boundary.py::TestScipyCompatibilityBoundary::test_scipy_
290. tests/test_scipy_compatibility_boundary.py::TestScipyCompatibilityBoundary::test_top_le
291. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_direct_imp
292. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_frozen_nor
293. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_frozen_nor
294. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_imported_s
295. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_new_normal
296. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_preexistin
297. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_real_scipy
298. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_scipy_entr
299. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_top_level_
300. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_uncertain_
301. tests/test_scipy_normal_compatibility.py::TestScipyNormalCompatibility::test_vectorized
302. tests/test_semantic_boundary.py::TestSemanticBoundary::test_semantic_module_owns_solve_
303. tests/test_semantic_boundary.py::TestSemanticBoundary::test_semantic_solve_module_does_
304. tests/test_semantic_solve_regression.py::TestSemanticSolveRegression::test_componentwis
305. tests/test_semantic_solve_regression.py::TestSemanticSolveRegression::test_interval_sem
306. tests/test_semantic_solve_regression.py::TestSemanticSolveRegression::test_polynomial_s
307. tests/test_semantic_solve_regression.py::TestSemanticSolveRegression::test_predictor_co
308. tests/test_semantic_solve_regression.py::TestSemanticSolveRegression::test_residual_int
309. tests/test_shared_generators.py::TestSharedGeneratorRepresentation::test_elementary_fun
310. tests/test_shared_generators.py::TestSharedGeneratorRepresentation::test_generator_matr
311. tests/test_shared_generators.py::TestSharedGeneratorRepresentation::test_path_aware_exe
312. tests/test_shared_generators.py::TestSharedGeneratorRepresentation::test_public_represe
313. tests/test_shared_generators.py::TestSharedGeneratorRepresentation::test_recomputed_exp
314. tests/test_shared_generators.py::TestSharedGeneratorRepresentation::test_sampled_nonlin
315. tests/test_shared_generators.py::TestSharedGeneratorRepresentation::test_semantic_solve
316. tests/test_shared_generators.py::TestGeneratorAlgebraDirect::test_manual_shared_generat
317. tests/test_spd_matrix_high_precision.py::TestSPDMatrixHighPrecision::test_rbf_like_spd_
318. tests/test_spd_matrix_high_precision.py::TestSPDMatrixHighPrecision::test_spd_enclosure
319. tests/test_spd_matrix_high_precision.py::TestSPDMatrixHighPrecision::test_two_by_two_sp
320. tests/test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_application_context_sp
321. tests/test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_mle_like_family_certif
322. tests/test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_non_symmetric_family_f
323. tests/test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_quad_solve_spd_does_no
324. tests/test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_separately_constructed

325. tests/test_spd_matrix_semantics.py::TestSPDMatrixSemantics::test_slogdet_quad_solve_and
326. tests/test_special_functions.py::TestTrustedSpecialFunctions::test_erf_interval_is_monc
327. tests/test_special_functions.py::TestTrustedSpecialFunctions::test_erf_points_contain_r
328. tests/test_special_functions.py::TestTrustedSpecialFunctions::test_normal_point_functic
329. tests/test_special_functions.py::TestSpecialExecution::test_erf_polynomial_dense_contai
330. tests/test_special_functions.py::TestSpecialExecution::test_interval_representation
331. tests/test_special_functions.py::TestSpecialExecution::test_math_erf_dispatch
332. tests/test_special_functions.py::TestSpecialExecution::test_parameterized_normal_helper
333. tests/test_special_functions.py::TestSpecialExecution::test_standard_normal_cdf_polynom
334. tests/test_special_functions.py::TestSpecialExecution::test_standard_normal_pdf_polynom
335. tests/test_split_scoring.py::TestCheapSplitScore::test_cheap_controller_certifies_only_
336. tests/test_split_scoring.py::TestCheapSplitScore::test_cheap_controller_matches_referen
337. tests/test_split_scoring.py::TestCheapSplitScore::test_cheap_controller_supports_genera
338. tests/test_split_scoring.py::TestCheapSplitScore::test_default_subdivision_policy_uses_
339. tests/test_split_scoring.py::TestCheapSplitScore::test_hidden_monotone_score_predicts_r
340. tests/test_split_scoring.py::TestCheapSplitScore::test_invalid_split_method_is_typed_pu
341. tests/test_split_scoring.py::TestCheapSplitScore::test_stationary_score_predicts_local_
342. tests/test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_elementary_gen
343. tests/test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_general_recurr
344. tests/test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_midpoint_preco
345. tests/test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_multiple_right
346. tests/test_step5_general_solve.py::TestStep5GeneralPolynomialSolve::test_quadratic_2x2_
347. tests/test_trusted_array_elementary.py::TestTrustedArrayElementary::test_elementwise_ar
348. tests/test_trusted_array_elementary.py::TestTrustedArrayElementary::test_exp_interval_a
349. tests/test_trusted_array_elementary.py::TestTrustedArrayElementary::test_exp_point_arra
350. tests/test_trusted_array_elementary.py::TestTrustedArrayElementary::test_integer_cache_
351. tests/test_trusted_array_elementary.py::TestTrustedArrayElementary::test_sqrt_point_arr
352. tests/test_trusted_boundary.py::TestTrustedBoundary::test_proof_critical_nextafter_is_c
353. tests/test_trusted_boundary.py::TestTrustedBoundary::test_trusted_modules_do_not_import
354. tests/test_trusted_elementary.py::TestTrustedElementary::test_edge_point_values
355. tests/test_trusted_elementary.py::TestTrustedElementary::test_interval_extensions_conta
356. tests/test_trusted_elementary.py::TestTrustedElementary::test_pi_and_ln2_constants_cont
357. tests/test_trusted_elementary.py::TestTrustedElementary::test_random_points_against_hig
358. tests/test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_certificate_
359. tests/test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_integer_inte
360. tests/test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_interval_lin
361. tests/test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_nonnegative_
362. tests/test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_point_reduct
363. tests/test_trusted_kernel_regression.py::TestTrustedKernelRegression::test_scalar_binar
364. tests/test_trusted_power.py::TestTrustedPower::test_integer_negative_base_and_domain_fa
365. tests/test_trusted_power.py::TestTrustedPower::test_interval_powers_contain_samples
366. tests/test_trusted_power.py::TestTrustedPower::test_point_powers_against_high_precision
367. tests/test_trusted_power.py::TestTrustedPower::test_random_positive_point_powers
368. tests/test_typed_failures.py::TestTypedFailures::test_all_public_execution_failures_sha
369. tests/test_typed_failures.py::TestTypedFailures::test_bad_representation_is_invalid_inp
370. tests/test_typed_failures.py::TestTypedFailures::test_empty_input_box_is_invalid_input

371. tests/test_typed_failures.py::TestTypedFailures::test_interval_semantic_solve_contracti
372. tests/test_typed_failures.py::TestTypedFailures::test_negative_power_zero_crossing_is_c
373. tests/test_typed_failures.py::TestTypedFailures::test_policy_validation_is_typed
374. tests/test_typed_failures.py::TestTypedFailures::test_public_backend_environment_failur
375. tests/test_typed_failures.py::TestTypedFailures::test_public_interval_validation_is_typ
376. tests/test_typed_failures.py::TestTypedFailures::test_uncertain_exponent_is_unsupported
377. tests/test_typed_failures.py::TestTypedFailures::test_unsupported_transcendental_is_typ
378. tests/test_typed_failures.py::TestTypedFailures::test_user_model_failure_is_not_mislabe
379. tests/test_typed_failures.py::TestTypedFailures::test_zero_crossing_division_is_certifi
380. tests/test_v1_adversarial_ht.py::TestV1AdversarialHT::test_reciprocal_proof_failure_tri
381. tests/test_v1_adversarial_ht.py::TestV1AdversarialHT::test_strict_closed_endpoint_prune

Appendix B. File-level count checksum

Test file	Named tests
test_application_package_boundary.py	2
test_application_theorems.py	8
test_array_semantics.py	16
test_batched_triangular_interval.py	4
test_branching_boundary.py	3
test_configuration_controller.py	16
test_controller_boundary.py	3
test_controller_context.py	2
test_controller_regression.py	4
test_domain_proofs.py	10
test_elementary_boundary.py	2
test_elementary_execution.py	14
test_enclosure_boundary.py	3
test_enclosure_regression.py	10
test_engineering_validation_corpus.py	5
test_external_engineering_validation.py	5
test_generator_aware_solve.py	6
test_generator_boundary.py	2
test_implicit_solve_semantics.py	16
test_input_specification.py	17
test_interval_reduction_performance.py	7
test_local_semantic_dispatch.py	9
test_migration_retirement.py	3
test_packed_polynomial_bernstein.py	8
test_packed_polynomial_monotonicity.py	7
test_packed_polynomial_ranging.py	6
test_parallel_accelerators.py	10
test_path_aware_branching.py	13
test_performance_kernels.py	5
test_polynomial_boundary.py	2

Test file	Named tests
test_polynomial_regression.py	8
test_power_execution.py	9
test_prepared_manybox.py	10
test_public_contract.py	3
test_reductions_exact.py	7
test_reference_regression.py	1
test_regular_grid_interpolator_semantics.py	13
test_release_hardening.py	10
test_scalar_binary64_exact.py	8
test_scipy_compatibility_boundary.py	3
test_scipy_normal_compatibility.py	11
test_semantic_boundary.py	2
test_semantic_solve_regression.py	5
test_shared_generators.py	8
test_spd_matrix_high_precision.py	3
test_spd_matrix_semantics.py	6
test_special_functions.py	9
test_split_scoring.py	7
test_step5_general_solve.py	5
test_trusted_array_elementary.py	5
test_trusted_boundary.py	2
test_trusted_elementary.py	4
test_trusted_kernel_regression.py	6
test_trusted_power.py	4
test_typed_failures.py	12
test_v1_adversarial_ht.py	2
Total	**381**